

How does the implementation of n logical constraints affect the search space and runtime of Grover's Algorithm, investigated in the context of simplified molecular library search?

Abstract

Grover's Algorithm is a quantum search algorithm that offers a theoretical quadratic speedup for unstructured search problems where no faster classical alternative exists. However, the speedup efficiency is highly sensitive to the number of logical constraints applied, which can reduce the valid search space or complicates the Oracle's constructions, leading to potential over-rotation, which is known as a phenomenon where applying too many iterations rotates the quantum state vector past the target solution, or increased circuit depth. This study investigates the implementation of n logical constraints and their effect on the space and runtime of Grover's Algorithm in the context of molecular library. For that purpose, this study used a dataset that contains 16 molecules used in drug discovery and biochemical research. Precisely, each molecule was assigned a unique 4-bit code based on their 4 specific properties: toxicity, the presence of a ring, the molecular size, and nitrogen presence. A 4-qubit circuit in Qskit, custom Oracle, and Diffuser were built using Python. The present research aims to explore the threshold where increasing logical constraints transitions from optimizing the search to degrading success probability. Our findings indicate that while increasing the number of n logical constraints effectively narrows the search space, it requires a precise number of Grover iterations (k) to maintain accuracy. We observed that if we go beyond this number, over-rotation occurs; the chance of finding the target solution begins to drop, making the final result much less clear. Ultimately, this demonstrates the potential for quantum searching to assist in identifying viable drug candidates in simplified chemical scenarios.

Key words: Quantum Algorithms, Grover's Algorithm, Quantum Search Optimization, Molecular Library Search, Logical Constraints, Quantum Speedup.

Introduction

The transition from classical to quantum computing represents a fundamental shift in how complex computational problems are approached. At the center of these changes is Grover’s Algorithm, a quantum search algorithm designed to address built-in inefficiencies of searching unstructured databases. In a classical framework, identifying a specific item within an unsorted list of size N requires a linear search strategy, resulting in an average time complexity of $O(N)$. Grover’s Algorithm, however, leverages the quantum mechanical principles of superposition and interference to provide a quadratic speedup, achieving the same result in $O(\sqrt{N})$ operations (Guo, 2023). As the field of quantum informatics matures, the focus has shifted from theoretical speedups to the practical implications of implementing these algorithms within specific domains, most notably in biological applications (Jura et al., 2025).

In the context of molecular search, the challenge is rarely to find an arbitrary item, but rather to identify a specific molecular candidate that satisfies a set of rigorous physical and chemical criteria. This necessity introduces the concept of “logical constraints” - specific Boolean conditions such as toxicity levels, molecular size, and elemental composition that act as filters within the search process. While Grover’s Algorithm is mathematically optimized for unstructured data, real-world chemical databases are increasingly viewed through the lens of constraint-aware systems. This study builds on prior implementations of Grover’s Algorithm in applied and experimental settings, extending these approaches to investigate how logical constraints interact with the quantum state vector. (Ayman et al., 2024).

The “search space” in a quantum search is identified by the Hilbert Space, where every possible solution is represented as a basis state in a uniform superposition. For a molecular library, this requires mapping chemical properties into binary representations, enabling structured mapping into quantum states. Prior work has shown that encoding problem-specific constraints directly into the initial state or oracle can significantly influence performance (Bae et al., 2026). In this study, molecules are represented using 4-bit strings corresponding to properties such as toxicity, molecular size, ring presence, and nitrogen content, allowing the search problem to be formalized within a 4-qubit system. A key parameter in Grover’s Algorithm is the number of valid solutions (M) relative to the total search space (N). The optimal number of iterations (k) is given by

$$k \approx \frac{\pi}{4} \sqrt{\frac{N}{M}}$$

Indicating that the runtime depends on the ratio of valid solutions to total states. As M increases, fewer iterations are required for successful amplitude amplification. Conversely, as logical constraints become stricter and reduce the number of valid solutions, the algorithm requires more iterations to amplify the correct state. This introduces a fundamental trade-off: while constraints

help isolate relevant candidates, they also increase the sensitivity of the algorithm to the precise number of iterations required for optimal performance.

Despite the theoretical advantage of Grover's quadratic speedup, significant barriers remain regarding Oracle complexity. The oracle is the functional block of the circuit responsible for recognizing and marking the target solutions. As logical constraints become more specific, the Oracle requires a deeper circuit with a higher number of quantum gates. On current NISQ (Noisy Intermediate-Scale Quantum) hardware, increased circuit depth leads to higher rates of decoherence and gate errors, which can degrade the final success probability (Hill, 2026). Recent studies emphasize that these limitations, specifically the resource - heavy nature of Oracle's construction, often act as a more significant bottleneck than the algorithm runtime itself.

Although Grover's Algorithm was not originally designed for structured or constraint-heavy problems, its applications to molecular systems has been explored in various contexts, including the Molecular Distance Geometry Problem (Lavor et al., 2016). These studies highlight both the promise and limitations of applying quantum search techniques to chemically relevant datasets.

This paper explores these dynamics by investigating how the implementation of n logical constraints affects the search space and runtime within a simplified molecular library. By analyzing the intersection of target density scaling and Oracle complexity, we aim to provide a clearer picture of the practical limits of quantum search in the burgeoning field of drug discovery. Through the analysis, we seek to determine at what point the overhead of logical constraints might outweigh the inherent speedup provided by Grover's Algorithm.

Methods

1.Dataset Construction and Encoding

This study utilizes a simplified molecular dataset consisting of 16 molecules commonly referred to in drug discovery and biochemical research. Each molecule was encoded as a unique 4-bit binary string, enabling direct representation within a 4-qubit quantum system.

Molecule Name	Bit 3: Toxicity (0=Safe, 1=Toxic)	Bit 2: Ring (0=No, 1=Yes)	Bit 1: Size (0=Small, 1=Large)	Bit 0: Nitrogen (0=No, 1=Yes)	Quantum State
Aspirin	0	1	1	0	0110
Benzene	1	1	0	0	1100
Caffeine	0	1	1	1	0111
Ethanol	0	0	0	0	0000
Nicotine	1	1	1	1	1111
Glucose	0	1	1	0	0110
Penicillin	0	1	1	1	0111
Formaldehyde	1	0	0	0	1000
Paracetamol	0	1	1	1	0111
Metformin	0	0	1	1	0011
Ibuprofen	0	1	1	0	0110
Dopamine	0	1	0	1	0101
Urea	0	0	0	1	0001
Cyanide	1	0	0	1	1001
Morphine	1	1	1	1	1111
Chloroform	1	0	0	0	1000

Table 1: Dataset representation.

The encoding was based on four predefined molecular properties: toxicity, presence of a ring structure, molecular size, and the presence of a nitrogen. Each property was treated as a binary variable (0 or 1), allowing every molecule to be mapped onto a computational basis state. This results in a complete representation of the dataset within a Hilber space if size $2^4 = 1$, transforming the molecular filtering problem into a Boolean search problem compatible with Grover’s Algorithm.

2.Quantum Circuit Design

All quantum circuits were implemented in Python using the Qiskit framework. The system consisted of four primary qubits representing molecular states, with additional auxiliary qubits introduced when required for Oracle construction.

The overall circuit followed the standard structure of Grover’s Algorithm, consisting of three main components: initialization, Oracle application, and amplitude amplification via the

Diffusion operator, followed by measurement. The circuit design was modular, allowing the Oracle component to be modified independently as the number of logical constraints increased.

3.State Initialization

The quantum system was initialized by applying Hadamard gates to all four qubits, generating a uniform superposition across all possible basis states. This ensures that each encoded molecule begins with equal probability amplitude prior to the application of any constraints.

4.Logical Constraint Implementation (Oracle Design)

Logical constraints were implemented through a custom Oracle designed to mark states that satisfy specified Boolean conditions. Each constraint corresponded to a requirement on one or more qubits, reflecting molecular properties such as nitrogen presence or molecular size.

The Oracle was constructed using combinations of single-qubit and multi-qubit gates. In particular, X gates were used to adjust qubit states to match the required logical conditions, while multi-controlled phase operators were applied to mark valid solutions through phase inversion. When multiple constraints were combined, multi-controlled gate structures were used to enforce conjunctions of conditions.

The number of constraints (n) was verified systematically by increasing the number of Boolean conditions encoded into the Oracle. As n increased, the Oracle required additional gate operations and, in some cases, auxiliary qubits to maintain correct logical behavior.

5.Diffusion Operator (Amplitude Amplification)

Following the Oracle, a diffusion operator was applied to amplify the amplitudes of the marked states. This operator was implemented using the standard Grover diffusion sequence, consisting of Hadamard transformations, Pauli-X gates, a multi-controlled phase inversion, and the inverse sequence of these operators.

The process performs an inversion about the mean of the amplitude distribution, increasing the probability of measuring states that satisfy the imposed constraints.

6.Iteration Strategy

The Grover Operator, defined as the combination of the Oracle and diffusion operator, was applied iteratively. The number of iterations (k) was varied depending on the number of valid solutions (M) resulting from the applied constraints.

For each configuration, multiple values of k were tested to examine how the probability distribution evolved. This approach allowed for the identification of iteration counts that maximize the likelihood of measuring valid states.

7.Simulation and Measurement

All circuits were executed using Qiskit's quantum simulator. After completing the specified number of Grover iterations, measurements were performed on all qubits in the computational basis.

Each circuit was executed over multiple runs (shots) to obtain a statistical distribution of outcomes. The resulting measurements data was used to determine the frequency with which constraint-satisfying states were observed.

8.Experimental Design

The experimental procedure involved systematically varying the number of logical constraints (n) applied within the Oracle. For each value of n , a corresponding Oracle was constructed and integrated into the Grover circuit.

The algorithm was then executed across a range of iteration counts (k), and measurement outcomes were recorded for each configuration. This structured approach enabled the investigation of how increasing constraints complexity influences the behavior of the algorithm within a fixed-size quantum system.

Results

1.Iterative Probability Amplification and State Isolation

The core mechanism of Grover's Algorithm is the amplification of the probability amplitude of a target state through constructive interference. This process is illustrated by the evolution of the probability distribution across Grover iterations.

After a single iteration ($k = 1$), the target state (0110) exhibits an increased probability relative to other states; however, a substantial portion of the probability mass remains distributed among non-target states. This indicates that the system has begun amplifying the correct solution but has not yet reached optimal separation.

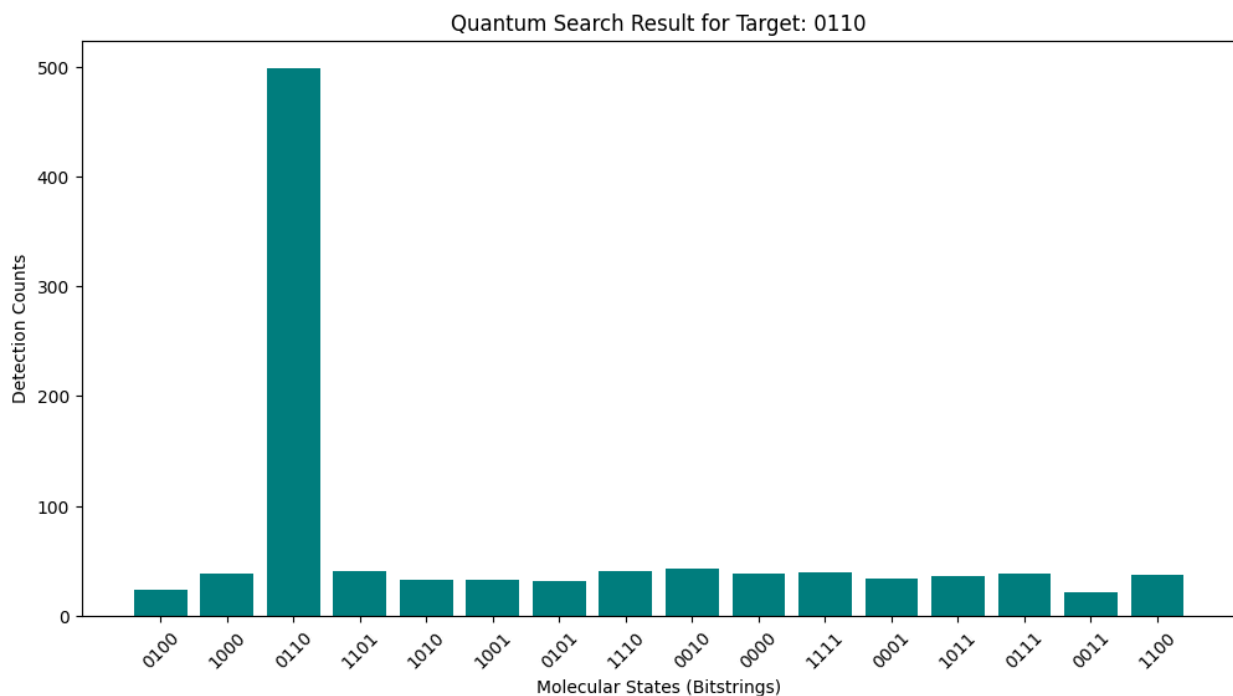


Figure 1: Probability distribution after one Grover iteration ($k = 1$) for a single target state (0110). The target state begins to show increased amplitude; however, significant probability remains distributed among non-target states, indicating incomplete amplification.

With additional iterations, the amplitude amplification becomes significantly more pronounced. At $k = 3$, the target state dominates the distribution, while non-target states are strongly suppressed. This demonstrates the convergence of the algorithm toward the optimal solution. Beyond this point, further iterations would lead to over-rotation, where the probability of the target state begins to decrease due to the oscillatory nature of amplitude amplification.

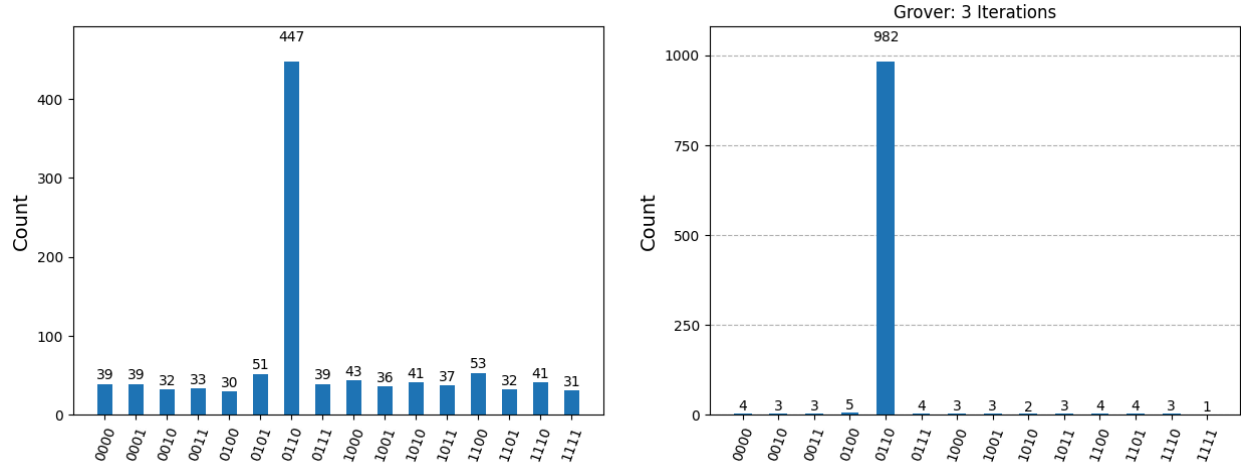


Figure 2: Comparison of probability distributions for $k = 1$ and $k = 3$ iterations. At $k = 3$, the target state (0110) reaches maximum amplification, while non-target states are suppressed, demonstrating optimal performance prior to over-rotation.

2.Impact of Logical Constraints and Solution Density

The efficiency of Grover's Algorithm is strongly dependent on the number of valid solutions (M) within the total search space ($N = 16$). As logical constraints are applied, M decreases, directly influencing the number of iterations required for optimal performance.

When searching for a single valid solution ($M = 1$), the algorithm requires a relatively large number of iterations to achieve maximum probability. This reflects the low initial density of target states in the superposition, requiring more amplification steps.

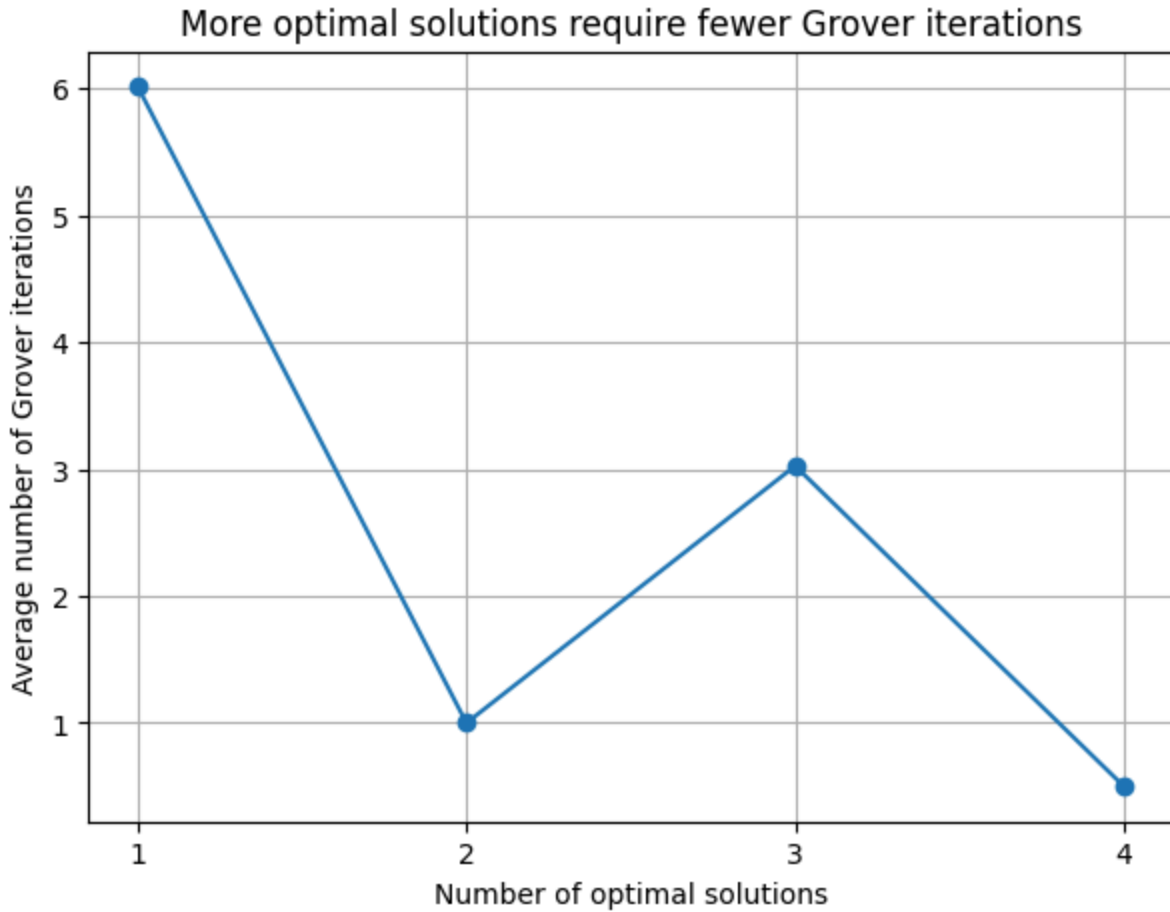


Figure 3: Effect of solution density (M) on optimal iteration count and success probability. Increasing the number of valid solutions reduces the number of required iterations, as a larger portion of the initial superposition corresponds to target states.

As the number of valid solutions increases, the required number of iterations decreases. For intermediate values of M (e.g., 2 or 3 solutions), the algorithm reaches high success probabilities with fewer iterations. This occurs because a larger proportion of the initial state already corresponds to valid solutions.

Iterations (k)	M=1 Success Rate	M=2 Success Rate
0	0.125	0.25
1	0.78125	1 (Optimal)
2	0.945312	0.25
3	0.330078	0.25
4	0.012207	1
5	0.547974	0.25
6	0.999786 (Optimal)	0.25
7	0.576973	1

8	0.019457	0.25
9	0.302891	0.25
10	0.931266	1

Table 2: Success Rate by Iteration for M=1 and M=2. This data reflects the probability of identifying target states as the iteration count (k) increases for single and double target configurations.

Iterations (k)	M=3 Success Rate	M=4 Success Rate
0	0.375	0.5
1	0.84375	0.5
2	0.023437	0.5
3	0.990234 (Optimal)	0.5
4	0.118652	0.5
5	0.677124	0.5
6	0.571381	0.5
7	0.19796	0.5
8	0.95719	0.5
9	0.001958	0.5
10	0.914383	0.5

Table 3: Success Rate by Iteration for M=3 and M=4. As the solution density increases, the algorithm exhibits periodic success and eventually reaches a plateau where success becomes balanced between valid and invalid states.

At higher solution densities (e.g., $M = 4$), the system approaches a regime where the probability distribution becomes more balanced. In this case, the algorithm exhibits reduced sensitivity to iteration count, and the maximum achievable probability is limited due to the symmetric distribution between valid and invalid states.

Number of Solutions (M)	Best Integer (k)	Plotted Average (k)	Maximum Success Rate
1	6	6.016578	0.999786
2	1	1	1
3	3	3.025896	0.990234

Table 4: Summary of Optimal Search Parameters. The summary below highlights the specific points where the algorithm achieves its peak theoretical and experimental efficiency.

3. Scaling Behavior and Comparison to Classical Search

To evaluate performance, the quantum search was compared to classical linear search for a fixed database size ($N = 16$). Classical search requires, on average, $O(N)$ operations, whereas Grover's Algorithm operates with $O(\sqrt{N})$ complexity.

The results demonstrate that the quantum implementation achieves the desired outcome in fewer iterations, consistent with theoretical predictions, compared to the classical average. This behavior is consistent with the theoretical quadratic speedup predicted for Grover's Algorithm.

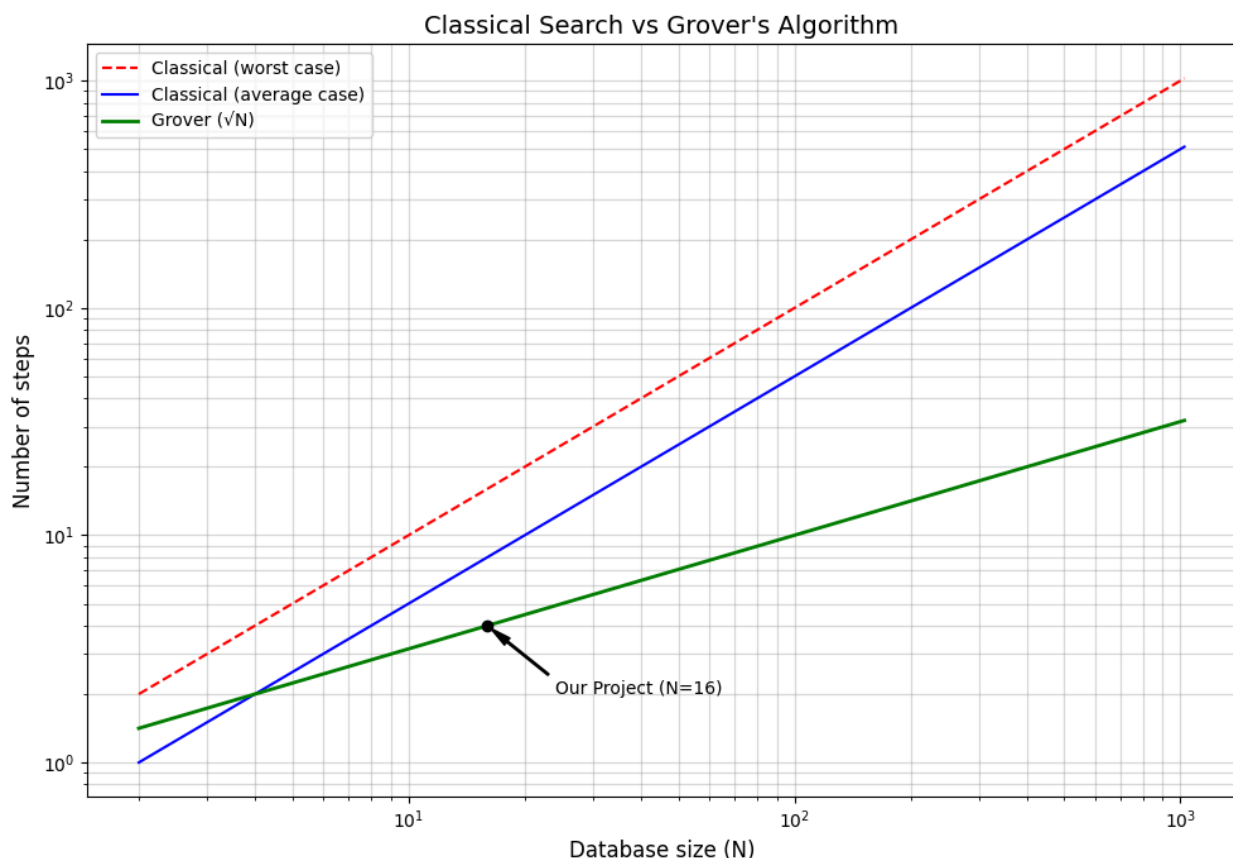


Figure 4: Comparison of classical and quantum search complexity. Classical search exhibits linear scaling ($O(N)$), while Grover's Algorithm follows quadratic speedup ($O(\sqrt{N})$), highlighting the increasing advantage of quantum search for larger datasets.

A complexity comparison plot further illustrates this difference, showing that as the size of the search space increases, the gap between classical and quantum performance widens. This suggests that quantum search methods become increasingly advantageous for larger and more complex molecular datasets.

Discussion

The results demonstrate that the introduction of logical constraints reduces the efficiency of Grover's Algorithm by decreasing the number of valid target solutions (M). As constraints become more restrictive, the density of valid states within the search space decreases, requiring a greater number of iterations to achieve successful amplitude amplification.

This finding highlights an important limitation in practical applications. While Grover's Algorithm offers a theoretical quadratic speedup, real-world problems, such as molecular screening, often involve multiple constraints that significantly reduce the number of acceptable solutions. In such cases, the increased iteration requirements and sensitivity to over-rotation may reduce the practical advantage over classical approaches. A key challenge identified in this study is the complexity of oracle design. As the number of constraints increases, the oracle requires more intricate logical structures, leading to increased circuit complexity.

This has two important implications. First, the development time required to construct a functional oracle grows with problem complexity. Second, in practical quantum computing environments, deeper circuits introduce greater susceptibility to noise and decoherence, reducing reliability. The results reveal a fundamental trade-off between constraint precision and algorithm performance. Increasing the number of constraints improves the specificity of the search, allowing more refined candidate identification. However, this comes at the cost of reduced solution density and increased sensitivity to the number of iterations.

As observed in the results, over-rotation effects become more pronounced when M is small, making precise selection of iteration count essential for maintaining high success probability. Future research could explore the use of weighted constraints, where different conditions are assigned varying levels of importance. In practical molecular screening, certain properties, such as toxicity, may act as strict requirements, while others function as secondary criteria.

Incorporating such weighting into Grover's framework could provide a more realistic model of applied search problems. This study was conducted using classical simulation rather than execution on quantum hardware. As a result, the findings do not account for noise, decoherence, or gate errors present in real systems.

Additionally, the experiment was limited to small qubit systems. While this allows clear observation of algorithmic behavior, further research is required to determine how these effects scale with larger datasets.

References

1. Guo, Chenxi. (2023). Grover's Algorithm – Implementations and Implications. Highlights in Science, Engineering and Technology. 38. 1071-1078. 10.54097/hset.v38i.5997.
2. Ayman, Judy., Hakeem, Manal. (2024). Grover's Algorithm. <https://github.com/ThinkingBeyond/IQRG-2024>
3. Lavor, Carlile & Liberti, Leo & Maculan, Nelson. (2016). Grover's Algorithm applied to the Molecular Distance Geometry Problem. 1-4. 10.21528/CBRN2005-234.
4. Hill, D. R. C. (2026). Grover Quantum Algorithm: Applications and Limits. Encyclopedia, 6(4), 89. <https://doi.org/10.3390/encyclopedia6040089>
5. Jura, A. M. C., Jura, Ș. A., Popescu, D. E., Belengeanu, V., Gușiță, B., Pienar, C., Manea, A. M., & Boia, E. R. (2025). Quantum Leap: Reshaping Genetic Diagnostics Using Quantum Computing. Preprints. <https://doi.org/10.20944/preprints202502.1371.v1>
6. Bae, E., Shin, J., & Choi, M. (2026). *Reducing circuit resources in Grover's algorithm via constraint-aware initialization*. arXiv. <https://doi.org/10.48550/arXiv.2601.17725>